

Introduction

In this practical session, you will continue SQL practice using the `classicmodels` database from TP06. You will write queries that span multiple tables, sort and filter results, and summarise data with aggregate functions.

Learning Objectives

- Retrieve data from more than one table using `JOIN`
- Remove duplicates with `DISTINCT`
- Sort results with `ORDER BY`
- Summarise data with `COUNT`, `SUM`, `AVG`, `MAX`, `MIN`
- Group results with `GROUP BY` and filter groups with `HAVING`

Submission Guidelines

- Write one query for each task
- End each statement with a semicolon
- Use clear indentation for multi-line queries
- Always specify which database to use at the start of the session

Database Reference

All exercises use `classicmodels`. Start every session with:

```
USE classicmodels;
```

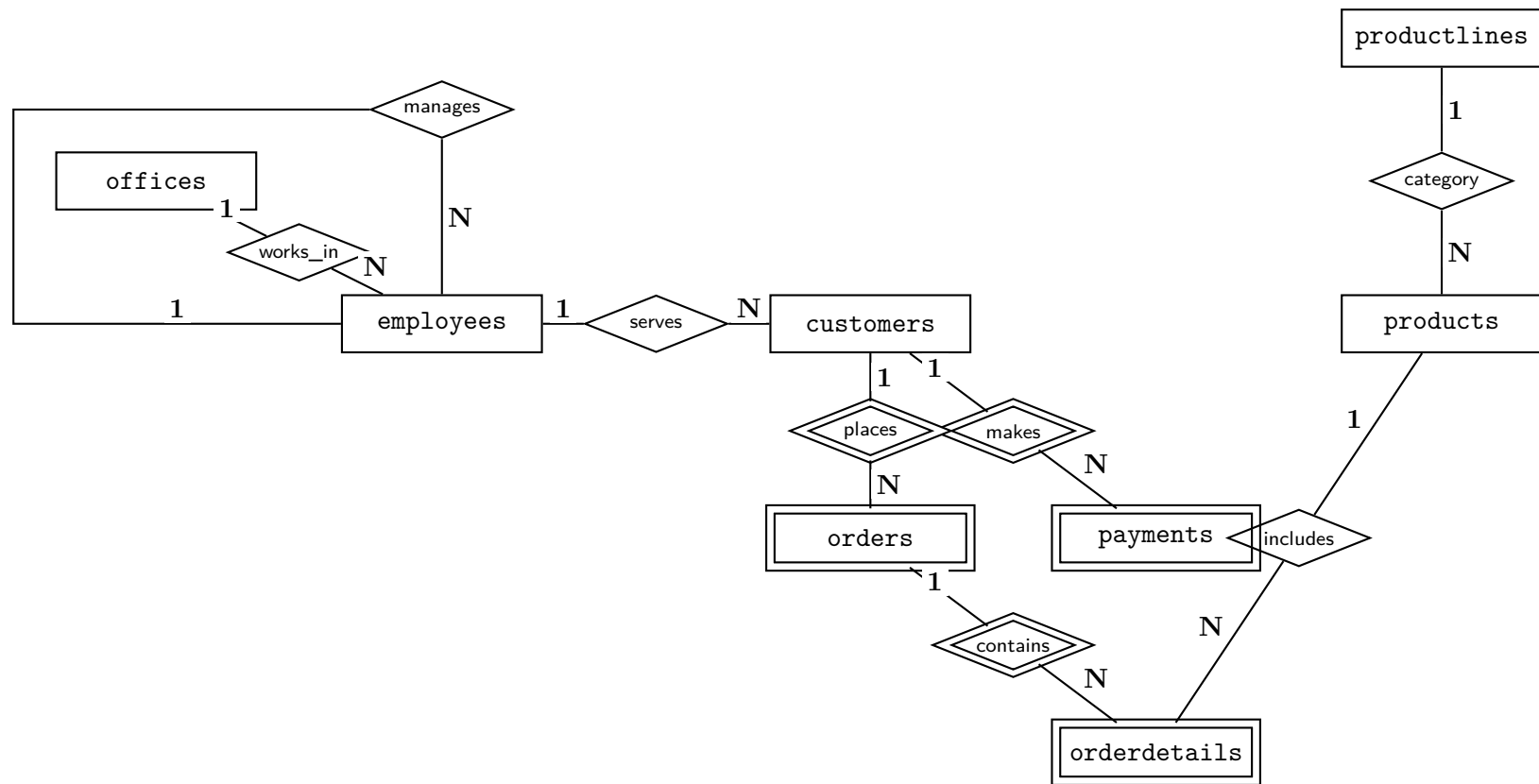
The relevant tables and their key columns are:

Table	Key Columns
<code>customers</code>	<code>customerNumber</code> , <code>customerName</code> , <code>city</code> , <code>country</code> , <code>creditLimit</code> , <code>salesRepEmployeeNumber</code>
<code>employees</code>	<code>employeeNumber</code> , <code>firstName</code> , <code>lastName</code> , <code>jobTitle</code> , <code>officeCode</code> , <code>reportsTo</code>
<code>offices</code>	<code>officeCode</code> , <code>city</code> , <code>country</code> , <code>phone</code>
<code>orders</code>	<code>orderNumber</code> , <code>orderDate</code> , <code>status</code> , <code>customerNumber</code>
<code>orderdetails</code>	<code>orderNumber</code> , <code>productCode</code> , <code>quantityOrdered</code> , <code>priceEach</code>
<code>payments</code>	<code>customerNumber</code> , <code>checkNumber</code> , <code>paymentDate</code> , <code>amount</code>
<code>productlines</code>	<code>productLine</code> , <code>textDescription</code>
<code>products</code>	<code>productCode</code> , <code>productName</code> , <code>productLine</code> , <code>quantityInStock</code> , <code>buyPrice</code> , <code>MSRP</code>

Relationships to remember from TP06:

From Table	Foreign Key		To Table	Cardinality
<code>customers</code>	<code>salesRepEmployeeNumber</code>	→	<code>employees.employeeNumber</code>	N:1
<code>employees</code>	<code>officeCode</code>	→	<code>offices.officeCode</code>	N:1
<code>employees</code>	<code>reportsTo</code>	→	<code>employees.employeeNumber</code>	N:1
<code>orders</code>	<code>customerNumber</code>	→	<code>customers.customerNumber</code>	N:1
<code>orderdetails</code>	<code>orderNumber</code>	→	<code>orders.orderNumber</code>	N:1
<code>orderdetails</code>	<code>productCode</code>	→	<code>products.productCode</code>	N:1
<code>payments</code>	<code>customerNumber</code>	→	<code>customers.customerNumber</code>	N:1
<code>products</code>	<code>productLine</code>	→	<code>productlines.productLine</code>	N:1

ERD Overview (Chen notation: rectangle = strong entity, double rectangle = weak entity, diamond = relationship, double diamond = identifying relationship)



Part 1 – JOIN Queries

Exercise 1 – INNER JOIN

Write SQL queries to:

1. Show each employee's full name (`firstName`, `lastName`), their `jobTitle`, and the `city` of their office. Use table aliases `e` and `o`.
2. Show `orderNumber`, `orderDate`, `customerName`, and `country` for all **Shipped** orders. *(2 tables, filter with WHERE)*
3. Show `customerName`, `orderNumber`, `productName`, `quantityOrdered`, and `priceEach` for every line item of every order placed by customers in 'France'. *(4 tables: customers, orders, orderdetails, products)*

Exercise 2 – LEFT JOIN and Missing Data

A `LEFT JOIN` keeps all rows from the left table even when there is no match.

1. Show all customers and their orders. Include customers who have **never placed an order** (their order columns should appear as `NULL`).
Hint: customers LEFT JOIN orders
2. From the result above, modify the query to show **only** the customers who have never placed an order.
Hint: add WHERE orders.orderNumber IS NULL
3. Show all products that have **never appeared** in any order.
Hint: products LEFT JOIN orderdetails

Exercise 3 – Self JOIN

The `employees` table references itself through `reportsTo`.

1. Show each employee's full name and their manager's full name. Use aliases `e` (employee) and `m` (manager).
2. Extend the query: also show the `city` of the office where the employee works. *(3 tables: employees self-join + offices)*
3. Show only employees whose manager's `jobTitle` is 'VP Sales'.

Part 2 – Sorting, Filtering, and Computed Columns

Exercise 4 – ORDER BY and DISTINCT

1. List all distinct countries that appear in `customers`, sorted A to Z.
2. List the **top 5** most expensive products by MSRP, showing `productName`, `productLine`, and MSRP.
Hint: ORDER BY ... DESC LIMIT 5
3. Show the 3 orders with the **earliest** `orderDate`.
4. List all products ordered first by `productLine` (A to Z), then by `buyPrice` (highest to lowest) within each line.

Exercise 5 – Computed Columns and CASE WHEN

A computed column is created inside `SELECT` using an expression.

1. For each row in `orderdetails`, calculate the **line total** as `quantityOrdered * priceEach`. Label the column `lineTotal`.
2. For each row in `products`, calculate the **profit margin** as `MSRP - buyPrice`. Show `productName` and `margin`, sorted by `margin` descending.
3. Use `CASE WHEN` to label each product by price:
 - `buyPrice < 30` → 'Budget'
 - `buyPrice between 30 and 80` → 'Mid-range'
 - `buyPrice > 80` → 'Premium'

Show `productName`, `buyPrice`, and the label column named `priceCategory`.

Part 3 – Aggregates, GROUP BY, and HAVING

Exercise 6 – GROUP BY with JOIN

1. Show each `customerName` and the **total number of orders** they have placed. Include customers with zero orders (use `LEFT JOIN`). Sort by order count descending.
2. Show each `productName` and the **total quantity ordered** across all orders. Sort by total quantity descending.
3. Show each `officeCode` (city), the number of employees in that office, and the number of customers those employees manage in total.
(*joins: offices → employees → customers, two COUNTs*)
4. For each order, calculate the **total order value** (`SUM(quantityOrdered * priceEach)`). Show `orderNumber` and `orderTotal`, sorted by `orderTotal` descending.

Exercise 7 – HAVING with Logic

1. Show countries with **more than 5** customers, along with the customer count. Sort by count descending.
2. Show product lines whose average `buyPrice` is above 60 **and** that contain **more than 10** products.
3. Show customers whose **total payment amount exceeds 100 000**. Display `customerNumber`, `customerName`, and total amount. (*join payments → customers*)
4. Show the **single order** with the highest total value.
Hint: use ORDER BY and LIMIT 1 after grouping.

Part 4 – Subqueries and Logic

A **subquery** is a query nested inside another query. It is written inside parentheses and can appear in **WHERE**, **FROM**, or **SELECT**.

Exercise 8 – Subquery in WHERE

1. Show all products whose **buyPrice** is **above the average** **buyPrice** of all products.
Hint: `WHERE buyPrice > (SELECT AVG(buyPrice) FROM products)`
2. Show all customers who are located in the same country as the customer named 'Signal Gift Stores'.
Hint: use a subquery to find the country first.
3. Show all orders placed **after** the most recent order placed by customer number 103.
Hint: subquery finds `MAX(orderDate)` for that customer.

Exercise 9 – Business Logic Queries

These questions require you to combine several concepts. Read each question carefully before writing the query.

1. **Sales rep performance:** For each sales representative (employee), show their full name and the **total number of customers** they manage. Show only reps who manage **at least 3** customers.
2. **Order value ranking:** Show each order's **orderNumber**, **customerName**, and total order value. Display only orders with a total value **greater than 5 000**. Sort by value descending.
3. **Inventory alert:** Show all products where **quantityInStock** is **less than the total quantity already ordered** for that product across all orders. These are products at risk of stock shortage.
Hint: subquery computes `SUM(quantityOrdered)` per `productCode` from `orderdetails`.
4. **Best product per line:** For each **productLine**, show the **productName** with the **highest MSRP**.
Hint: subquery finds `MAX(MSRP)` per product line, then the outer query matches it.

Expected Outcome

After completing this TP, students should be able to:

- write `INNER JOIN` and `LEFT JOIN` across two, three, or four tables
- detect missing data with `LEFT JOIN ... IS NULL`
- perform self joins and multi-level joins
- build computed columns and use `CASE WHEN` for labelling
- combine `GROUP BY`, `HAVING`, and `JOIN` in one query
- write subqueries in `WHERE` to answer comparative and filtering questions
- solve real business logic problems using multi-step SQL reasoning

Reference – JSON Aggregation in MySQL 8.0+

MySQL 8.0 introduces two aggregate functions that output JSON directly from query results: `JSON_ARRAYAGG` and `JSON_OBJECTAGG`. They are useful for building API-ready payloads without extra application code.

Sample Tables

```
CREATE TABLE products (  
  id          INT PRIMARY KEY,  
  name        VARCHAR(50),  
  category    VARCHAR(50)  
);  
  
CREATE TABLE reviews (  
  id          INT PRIMARY KEY,  
  product_id  INT,  
  reviewer    VARCHAR(50),  
  rating      INT  
);  
  
INSERT INTO products VALUES  
  (1, 'Wireless_Mouse', 'Electronics'),  
  (2, 'Running_Shoes', 'Apparel');  
  
INSERT INTO reviews VALUES  
  (1, 1, 'Alice', 5),  
  (2, 1, 'Bob', 4),  
  (3, 2, 'Charlie', 5);
```


1. JSON_ARRAYAGG – collect values into a JSON array

Groups row values into a single JSON array per group.

```
SELECT
    p.name,
    JSON_ARRAYAGG(r.reviewer) AS reviewer_list
FROM products p
LEFT JOIN reviews r ON p.id = r.product_id
GROUP BY p.id, p.name;
```

name	reviewer_list
Wireless Mouse	["Alice", "Bob"]
Running Shoes	["Charlie"]

2. JSON_OBJECTAGG – build a JSON object from two columns

Pairs a key column and a value column into a JSON object per group.

```
SELECT
    p.name,
    JSON_OBJECTAGG(r.reviewer, r.rating) AS review_scores
FROM products p
LEFT JOIN reviews r ON p.id = r.product_id
GROUP BY p.id, p.name;
```

name	review_scores
Wireless Mouse	{"Bob": 4, "Alice": 5}
Running Shoes	{"Charlie": 5}

3. Nesting Both – complex JSON payloads

You can nest `JSON_ARRAYAGG` inside `JSON_OBJECTAGG` to build structured, API-ready output directly from the database.

```
SELECT
  p.category,
  JSON_OBJECTAGG(
    p.name,
    (SELECT JSON_ARRAYAGG(rating)
     FROM reviews
     WHERE product_id = p.id)
  ) AS category_report
FROM products p
GROUP BY p.category;
```

category	category_report
Electronics	{"Wireless Mouse": [5, 4]}
Apparel	{"Running Shoes": [5]}

Key points to remember:

- Both functions require `GROUP BY` to define the grouping scope.
- `JSON_ARRAYAGG` takes **one** argument (the value column).
- `JSON_OBJECTAGG` takes **two** arguments: key, then value. Keys must be unique within each group.
- Both are available in **MySQL 8.0+** only.
- `NULL` values in the key column will cause an error in `JSON_OBJECTAGG` – use `COALESCE` to handle them safely.